GENERATING RISK-CRITICAL SCENARIOS FOR AUTONOMOUS HIGHWAY LANE CHANGING VIA MULTI-AGENT SOFT ACTOR-CRITIC

マルチエージェントSoft Actor-Criticを用いた自動運転の高速道路車線変更におけるリスククリティカルシナリオ生成

by

ALTEZA Gabriel Pascua

Department of Mechanical & Aerospace Engineering
School of Engineering

Degree thesis is submitted for

Supervisors:
Dr. Tatsuya Suzuki
Dr. Hiroyuki Okuda

Nagoya University
August, 2026

Contents

0.1 Abstract

The lane-changing task is notoriously one of the more difficult to generate dynamic, risk-critical testing scenarios due to its

No known industry standard on lane

0.2 Introduction

The deployment of autonomous driving technologies into society has been one of the most delayed advancements to the field of autonomous vehicles, especially dealing with human pushback [**?**]. As such, researchers and scientists have been working towards developing various methods to test the safety and robustness of autonomous driving (AD) tasks on the road.

One of the least-explored AD task in terms of safety testing is the lane-changing or lane-intrusion AD task in which the vehicle tries to insert itself within a specific gap. Past papers have employed both static methods of evaluating the ability of a vehicle's lane changing, especially with regard to global standards, such as the ISO. However, there remains a huge gap in the available research on dynamic methods wherein obstacles are moving or actively working against the system under test (SUT). As such, this research aims to create a safety verification system for the lane-changing or lane-intrusion task into the gap of a leading-following pair of vehicles. Moreover, this research will test the lane-changing logic's ability to withstand multiple lane changes, coming in from a merging ramp and traversing the highway to reach the preset desired exit ramp situated on the other edge of the highway.

Reinforcement learning, more broadly, machine learning, has been a hot area of research for its efficacy as a research methodology tool in solving problems that are not easily solvable through traditional white-box means. Reinforcement learning, which uses a neural network to learn from repeated simulations within a set environment, learns from its mistakes and successes in order to develop a policy that grows stronger and eventually converges to a robust neural network for the decision-making of an appropriate action to take within the environment under test.

In the field of autonomous driving safety testing, one of the most important parts to research on is the coverage of the tests, namely how diverse the testing scenarios generated are from one another in order to ensure that a wider scope of possible situations are covered. Moreover, rare cases in which risk of crashing is high are difficult to quantify and hard-code manually, while they remain to be the most important to be covered in the overall scope, especially since they are directly linked to human comfortability and safety on the road when the vehicles employing such are eventually deployed on the road.

Hence, this study aims to use specifically a multi-agent soft-actor-critic reinforcement learning framework to control a platoon of moving vehicles and let them learn how to get themselves as close as possible to inducing high risk of crash within the SUT without actually letting it crash. Moreover, the soft-actor-critic portion of this framework aims to maximize the entropy of the discovery of appropriate actions. Rather than outputting one proven action that works, the policy outputs a probability mapping of various non-clusterable actions that all efficiently maximize risk of

crash in the SUT. In such way, the coverage of test-case generation is hypothetically increased, and may be evaluated against previous baseline methods for its strength in generating high-risk edge cases for lane-changing and lane-intrusion scenarios on a highway.

0.2.1 Lane Changing Models

Kinematic Bicycle Model

The vehicles all will be modeled using the kinematic bicycle model (KBM). Based on the KBM model, the states and inputs to be considered are as follows.

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \\ \theta_{t+1} \\ v_{t+1} \\ \gamma_{t+1} \end{bmatrix} = \begin{bmatrix} x_t + \Delta t v_t \cos\theta_t \\ y_t + \Delta t v_t \sin\theta_t \\ \theta_t + \Delta t \frac{v_t}{L_b} \tan(\gamma_t) \\ v_t + \Delta t a_t \\ \gamma_t + \Delta t \omega_t \end{bmatrix} \tag{1}$$

Intelligent Driver Model (IDM) + Minimizing Overall Braking Induced by Lane Changes (MOBIL)

The IDM is a car-following model that describes the longitudinal dynamics of a vehicle based on its desired speed, and the distance to the vehicle in front of it. The IDM is defined by the following equation:

0.2.2 Reinforcement Learning

The field of reinforcement learning (RL) is a subfield of the broader field of machine learning focusing on training an agent to make the appropriate decisions in a particular environment. Based on the characteristics of the environment, a reward function is defined and is made to provide the agent with feedback on the appropriateness of its actions. The agent then uses this feedback to learn the appropriate policy to take in order to maximize the expected return.

Reinforcement learning has multiple ways of being implemented, with actors and critics being the backbone of its most popular implementations. The concept of learning by mistake is governed by learning how an environment works *criticrole* and how to best navigate it (actor role). Critic-only RL implementations, such as value-based methods, are utilized in situations wherein an environment is fully observable and has no room for ambiguity in intuitively choosing appropriate actions when the goal is clearly defined. As such, there is no need for learning the intricacies of an environment. Rather, the critic simply gives a score to what is currently observed in the environment in order to determine the best course of action.

Actor-only RL implementations, on the other hand, trains an agent how to act based on raw in environments where either the specific configuration of an action to take in the environment is complex or the environment is partially observable. They directly parameterize a policy (the actor) and update it by optimizing the expected return (using policy gradients)

Soft Actor-Critic Reinforcement Learning

Multi-Agent Reinforcement Learning

0.2.3 Farama Foundation Gymnasium Highway Environment Libraries

The Farama Foundation has been a long-time provider of extensive libraries for reinforcement learning research and simulations. The Highway-Env library is a library that is built off of the Gymnasium library, which is a library that provides a framework for creating customisable environments, as well as premade environments, such as the double pendulum and the cart-pole.

Typical Training Pipeline

Figure **??** displays the generalized training process that the Gymnasium library utilizes.

Upon initialization of the environment, one is provided with a pair of outputs, which consists of (1) the initial observation matrix, that quantifies the current state of the environment and its agents, and (2) info, which is a dictionary of additional information about the environment, used for debugging and analysis purposes.

With every step of the environment, the current observation matrix is fed into an arbitrary policy to receive a set of actions for which the agents of the environment are then tasked to enact. Using the actions from the arbitrary policy, they are then fed back into the environment to step the current state into the next state, which then provides a set of five outputs:

1. an observation matrix, that quantifies the next state of the environment and its agents;
2. a reward matrix, that represents the quantifiable results of an agent's actions;
3. a termination boolean, that indicates whether the episode has just been terminated
4. a truncation boolean, that indicates whether the episode has just been truncated
5. info, which is a dictionary of additional information about the updated environment

Typically, in the case of reinforcement learning, the aforementioned arbitrary policy is replaced with a policy that is undergoing training based on the information of the current observation matrix, next observation matrix, and the reward matrix that represents whether the actions taken by the agents were appropriate or not. The training process is repeated until the policy converges to a robust policy for use in testing and evaluation.

The Highway-Env library is a library that provides a set of pre-built environments that simulate various highway driving scenarios.

Highway-Env Environment Initialization

Highway-Env and Gymnasium are both Python class-based libraries that allow the creation of environments of any imaginable geometry for simulating various highway driving scenarios. Each object or entity in the environment is represented as a class, thereby providing a modular and object-oriented approach to the creation of road objects and entities that fit the needs of the researcher.

Upon initialization of the highway environment, one is asked to provide a set of configurations that define the how the observation and action spaces of the environment are executed. All environments are initialized with an observation space and an action space. The observation space is the input of the environment, which is a set of numbers that describe the current state of the environment. The action space is the output of the environment, which is a set of numbers that describe the possible actions that can be taken in the environment.

A "Built with the goal of machine learning training in mind, in every step of a simulation in a particular environment with Gymansium, the environment returns (1) an observation matrix, that quantifies the current state of the environment and its agents; (2) a reward matrix, that represents the results of the agent's actions; (3) a next-observation matrix, that represents the state of the environment after the agent's actions; (4) a boolean of whether the episode has terminated. and a boolean of

0.3 Objectives

The objective of this research is detailed as follows:

1. To create a multi-lane highway environment that simulates a highway-traversing scenario where the ego vehicle is tasked to conduct lane changes to reach the desired exit lane, while the other vehicles are tasked to bully the ego vehicle into high-risk scenarios without actually crashing into it.

2. To implement the baseline IDM + MOBIL model in an ego lane-changing vehicle's goal of reaching the desired exit

3. To implement a multi-agent soft actor-critic reinforcement learning framework to control the adversarial platoon

4. To evaluate the diversity and breadth of scope of scenario generation of the trained adversarial platoon

0.4 Methodology

0.4.1 Highway Environment

Important Configurations

The highway environment used a custom-made environment using the Python Gymnasium and Highway-Env libaries.

The highway environment of this research was a modified class of the AbstractEnv class of the Highway-Env library. In the customization of an AbstractEnv, the proponent was required to declare the configurations of the environment, as seen in Appendix A, which contains the type of Observation and Action spaces (i.e., input of the environment).

The action space of the highway environment was of the MultiAgentAction class, which is a two-dimensional action space that contains a total of n vehicles taking m actions, where n is the number of vehicles in the environment and m is the number of actions that each vehicle can take. The action space was designed to be discrete, with each vehicle having a total of 5 actions to choose from: ContinuousAction

Kinematics

The adversarial vehicles, hereafter referred to as the "adversaries," are controlled by a multi-agent soft actor-critic reinforcement learning agent, and the other vehicles are controlled by a predefined baseline behavior model, specifically the IDM + MOBIL model.

Road Geometry

The specifics of the environment are as follows:

- 2 cruising lanes, 4 meters wide each
- 1 merging lane, 4 meters wide
- 1 exit lane, 4 meters wide
- Cruising space of 300 meters
- 1 ego vehicle coming from merging lane, 2 adversarial vehicles coming from cruising lanes

Figure **??** shows the schematic of the highway environment used in this research. The blue vehicle is the ego lane-changing vehicle controlled by the IDM + MOBIL model, whereas the green vehicles are the adversarial vehicles controlled by MASAC-RL.

Observation Space

The observation space of the highway environment is the part of the reinforcement learning framework that provides the agent with the necessary numbers that describe its workable environment. As such, the observation space was designed in a way that balanced the need for necessary information that did not bloat the observation space with redundant details that may hinder the learning process of the agent.

The original observation space provided by the highway environment library is of the MultiAgentObservation class, which is a three-dimensional observation space that contains a total of n vehicles observing n vehicles (including itself) for six of its features:

1. Presence (i.e., the vehicle is on screen or not): 0 for not present, 1 for present

2. Absolute longitudinal distance of the vehicle from the start of the highway: measured in meters
3. Absolute lateral distance of the vehicle from the left boundary of the highway: measured in meters
4. Longitudinal speed of the vehicle: measured in meters per second
5. Lateral speed of the vehicle: measured in meters per second
6. Heading angle of the vehicles: measured in radians

The observation space was taken and modified to reduce space bloating for the agent training process.

0.4.2 MASAC-RL Framework

Each of the adversaries' actor and critic networks used the same shared parameters, especially as vehicles in the highway environment were set to be homogeneous in their behavior and dynamics.

The proponent of this research decided to transform the three-dimensional observation space of the highway environment into a flat one-dimensional observation space for use as input in the actor and critic networks of the MASAC-RL framework. The input space of the actor network consists of the following observations lifted from the raw values of the original observation space provided by the highway environment:

- Self-observation
    1. Activity status (i.e., crashed or not): 0 for not crashed, 1 for crashed
    2. Lateral distance from left boundary, normalized to the highway width: 0 1
    3. Lateral distance from right boundary, normalized to the highway width: 0 1
    4. Relative longitudinal distance from ego LC, normalized to the highway length: -1 1
    5. Relative lateral distance from ego LC, normalized to the highway width: -1 1
    6. Longitudinal speed, normalized to maximum allowable speed: 0 1
    7. Lateral velocity, normalized to $\frac{1}{\sqrt{2}}$ of the maximum allowable speed: 0 1
- Ego LC observation
    1. Longitudinal distance from the exit ramp, normalized to the highway length: 0 1
    2. Lateral distance from the exit ramp, normalized to the highway width: 0 1

3. Relative longitudinal speed, normalized to twice the maximum allowable speed: -1 1
4. Relative lateral speed, normalized to $\frac{2}{\sqrt{2}}$ the maximum allowable speed: -1 1
5. $1/(2\text{DTTC}+1)$ with the 0 1 main adversary vehicle: 0 1

- Other-adversary observation

1. Relative longitudinal distance from the main adversary vehicle, normalized to the highway length: -1 1
2. Relative lateral distance from the main adversary vehicle, normalized to the highway width: -1 1
3. Relative longitudinal speed from the main adversary vehicle, normalized to twice the maximum allowable speed: -1 1
4. Relative lateral speed from the main adversary vehicle, normalized to $\frac{2}{\sqrt{2}}$ the maximum allowable speed: -1 1
5. $1/(2\text{DTTC}+1)$ with the main adversary vehicle: 0 1

Each of the actor and critic networks were fed the observational inputs of all the vehicles in each step chosen for training, once per adversarial vehicle's unique observation.

Reward Function

The reward function of the highway environment, as mentioned, is the part of the reinforcement learning framework that details the motivation of the agents to learn a specific behavior. As such, the reward

$$R_{adv-adv} = \begin{cases} -50 & \text{if } \tau < 1.0 \\ -60\tau + 10 & \text{if } 1.0 \leq \tau < 4.0 \\ -2\tau + 24 & \text{if } 4.0 \geq \tau \end{cases} \tag{2}$$

$$R_{adv-ego} = \begin{cases} -60 & \text{if } \tau < 1.0 \\ -20.8(\tau - 2.8)^2 + 100 & \text{if } 1.0 \leq \tau < 4.0 \\ -6.25\tau + 15 & \text{if } 4.0 \geq \tau \end{cases} \tag{3}$$

If release phase (lane changer less than 20 meters away from exit):

$$R_{adv-ego} = -6.25\tau + 15 \tag{4}$$

$$R = -N + \frac{P+N}{1+\exp(-k_1(x-a))} - \frac{P}{1+exp(-k_2(x-b))} \tag{5}$$

0.5 Results & Discussion

0.6 Conclusion

0.6.1 Risk Evaluation Methods

0.7 Appendix

The pipeline of the lane-change maneuver of the ego vehicle involves the separation of the "if," "when," and "how" frameworks of the maneuver, as developed in [**?**]. The "if" pertains to determining whether or not a lane-change maneuver into the next lane is appropriate. The "when" part aims to determine which gap to intrude into and when to begin the move. Lastly, the "how" pertains to the usage of low-level controllers to execute the maneuver according to the determined appropriate gap.

LC Global Planner

In the LC, the global planner takes care of the "if" part by utilizing lane-utility values that correspond to appropriateness of a move into the said lane. The four utility values in this research are as follows:

1. Average travel time in the lane
2. Average time gap density in the lane
3. Remaining travel time in the lane
4. Urgency of reaching the lane in achieving end position

The four equations representing the above normalized lane-utility values $U_{\text{avetravtime}}$, $U_{\text{avetimegap}}$, $U_{\text{remtravtime}}$, U_{urgency} are detailed as follows:

$$d_{\max} = \beta v_{\text{ref}} \tag{6}$$

$$U_{\text{avetravtime}} = \frac{-\left|\frac{d_{\max}}{v_{\text{ref}}} - \frac{d_{\max}}{\max(\gamma, v_\mu)}\right|}{\left|\frac{d_{\max}}{v_{\text{ref}}} - \frac{d_{\max}}{\gamma}\right|} \tag{7}$$

$$U_{\text{avetimegap}} = \frac{\min(\alpha t_{\text{gap,desired}}, t_{\text{gap},\mu})}{\alpha t_{\text{gap,desired}}} \tag{8}$$

$$U_{\text{remtravtime}} = \frac{\frac{\min(d_{\max}, d_{\text{end}})}{v_{\text{ref}}}}{\frac{d_{\max}}{v_{\text{ref}}}} \tag{9}$$

$$U_{\text{urgency}} = \exp\left(-\kappa \frac{i_{\text{current}} + 1}{n_{\text{lanes}} + 1} \cdot \Delta s\right) \tag{10}$$

The four utility values are then consolidated as seen in Equation (**??**).

$$U_{\text{total}} = \sum_{i \in A} w_i U_i \tag{11}$$

$$A = \{\text{avetravtime}, \text{avetimegap}, \text{remtravtime}, \text{urgency}\} \tag{12}$$

The lane with the highest utility values is then chosen as the target/desired lane, as seen in Equation (**??**).

$$L_{\text{desired}} = \underset{l \in L}{\text{argmax}} \left(U_l - (1 + \zeta |l|) \, |U_0|\right) \tag{13}$$

The global planner is then made to choose the best inter-vehicle traffic gap in the desired lane and the time instance in which it shall enact the lane intrusion. To achieve such, a "time-position"-area, is formulated, corresponding to the intersection between the reachable set of the ego car given its dynamical limitations (current velocity, allowed acceleration values, etc.), and the predicted safe lane change region, which is defined as the space that isn't occupied by any vehicle currently and across the prediction horizon.

The "time-position"-areas across the discretized horizon are summed and accumulate into a final "gap utility" value, displayed as A_g. The gap with the largest gap utility value, as seen in Equation (**??**), is chosen.

$$\begin{aligned} A_g = \sum_{k=0}^{K} \max \Bigg(0, \min \Bigg(& x_{E_k}^{\max}, \min_{i \in F_g} (x_{i_k} - [t_{\text{gap,min}} \min(v_{x_{\max}}, v_{x_{i_k}}) + d_s]) \Bigg) \\ & - \max \Bigg(x_{E_k}^{\min}, \max_{i \in B_g} (x_{i_k} + [t_{\text{gap,min}} v_{x_{i_k}} + d_s]) \Bigg) \Bigg) \end{aligned} \tag{14}$$

- $t_{\text{gap,min}}$ is the minimum allowed time gap to the chosen vehicle
- $v_{x_{\max}}$ is the maximum allowed velocity for the ego
- x_{i_k} and $v_{x_{i_k}}$ refer to the predicted position and velocity values of the given vehicle, respectively
- d_s is a safe-distance buffer
- $x_{E_k}^{\max}$ and $x_{E_k}^{\min}$ are the highest and lowest reachable longitudinal distances for the ego, respectively

The minimum time (discretized) that it takes for a lane change maneuver must also be accounted for. and must require that the amount of discretized time that the gap utility value exists is greater than or equal to the said minimum discretized time, as seen in Equation (**??**)

$$k_{A_g}^{\text{end}} - k_{A_g}^{\text{start}} \geq k_{\min} \tag{15}$$

Moreover, the time instance at which the move is initiated is chosen by taking the minimum magnitude of the required control signals (e.g., acceleration/deceleration to reach the "time-position"-area starting from k_{Peri}.

$$k_{A_g}^{\text{start}} \leq k^{\text{Peri}} \leq K - k_{\min} \tag{16}$$

The critic network aims to critique the action that an entity takes. This is done by taking a set of data per iteration under review (initial state, action taken, reward received, and final state), taking the current state and action taken, running them through the critic network to receive an evaluatory Q value (i.e., experimental value), which is then compared with the Bellman-equation-derived Q value (i.e., theoretical value), calculated using the final states. The loss (i.e., error) is then calculated and used to determine how the weights in the neural network should be altered (via backpropagation).

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a') \tag{17}$$

Now, this process is done over a sample of iterations, not just a singular iteration under review.

$$Target = Reward + \gamma Critic(NextState's, Actor(NextState's)) \tag{18}$$

The total reward R_t at time step t is formulated as the sum of four distinct components representing Time-to-Collision (TTC) proximity pressure, goal node achievement, ego vehicle crash penalties, and adversarial vehicle crash penalties:

$$R_t = R_{\text{TTC}} + R_{\text{goal}} + R_{\text{ego\_crash}} + R_{\text{adv\_crash}} \tag{19}$$

1. Time-to-Collision (TTC) Squeeze Component

Let A be the set of all controlled adversarial vehicles. For each adversary $i \in A$, the Time-to-Collision based on polygon geometry is denoted as τ_i. The proximity reward term is calculated via the following piecewise function:

$$R_{\text{TTC}} = \sum_{i \in A} f(\tau_i) \tag{20}$$

where

$$f(\tau_i) = \begin{cases} \dfrac{4.0}{\tau_i + 0.1} & \text{if } 1.0 \leq \tau_i \leq 4.0 \\ -10.0 & \text{if } 0.0 \leq \tau_i < 1.0 \\ 0.0 & \text{otherwise} \end{cases} \tag{21}$$

2. Goal Achievement Component

Let L_{ego} represent the current structural lane index pair (u, v) of the ego vehicle within the road network, and let $C_{\text{ego}} \in \{0, 1\}$ be a binary indicator denoting whether the ego vehicle has crashed (1 for crashed, 0 for safe):

$$R_{\text{goal}} = \begin{cases} +50.0 & \text{if } L_{\text{ego}} = (\text{'}l\text{'}, \text{'}m\text{'}) \text{ and } C_{\text{ego}} = 0 \\ 0.0 & \text{otherwise} \end{cases} \tag{22}$$

3. Crash Guardrail Penalties

The severe negative constraints applied to prevent collision failures for both the ego vehicle ($R_{\text{ego\_crash}}$) and individual adversarial vehicles ($R_{\text{adv\_crash}}$ where $C_i \in \{0, 1\}$ marks the crash state of adversary i) are defined as:

$$R_{\text{ego\_crash}} = \begin{cases} -100.0 & \text{if } C_{\text{ego}} = 1 \\ 0.0 & \text{otherwise} \end{cases} \tag{23}$$

$$R_{\text{adv\_crash}} = \sum_{i \in A} g(C_i) \quad \text{where} \quad g(C_i) = \begin{cases} -50.0 & \text{if } C_i = 1 \\ 0.0 & \text{otherwise} \end{cases} \tag{24}$$

4. Unified Dense Form

Expressed compactly utilizing indicator function notation $\mathbb{I}(\cdot)$, the comprehensive multi-agent reward function evaluates to:

$$R_t = \sum_{i \in A} \left[\mathbb{I}(1 \leq \tau_i \leq 4) \frac{4}{\tau_i + 0.1} - 10 \cdot \mathbb{I}(0 \leq \tau_i < 1) - 50 \cdot C_i \right] + 50 \cdot \mathbb{I}\big(L_{\text{ego}} = (\text{'}l\text{'}, \text{'}m\text{'})\big)(1 - C_{\text{ego}}) - 100 \cdot C_{\text{ego}} \tag{25}$$

For the reinforcement learning part, a multi-agent SAC RL is used

So essentially what i wanna do is - use SAC RL to explore local extrema to look for specific coordinations - once converged enough(the actor loss, qf loss, and alpha), freeze the policy and use nondeterministic (probabilistic) to still output actions –> remember that the alpha has both mean and standard dev

the more you train the policies, the higher poss that standard dev gets smaller and smaller and essentially becomes deterministic (not the best for farming edge cases of same coordination style, different configuration)

can write script that recognizes when a policy has converged enough for it to find a specific coordination and then find s